Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Skip List

.

Design and Implementation of a -SKIP LIST-

Team Name: *Team #*	*Avengers*
Team member 1 (Team Lead)	*Gera, Sarthak – 2510380077* *2510380077@geu.ac.in* 
Team member 2	*Bhadana, Arjun – 2510370135* *2510370135@geu.ac.in* 
Team member 3	*Gera, Rudraksh – 2510350038* *2510350038@geu.ac.in* 
Team member 4	*Silswal,Divyansh – 2510370989* *2510370989@geu.ac.in* 

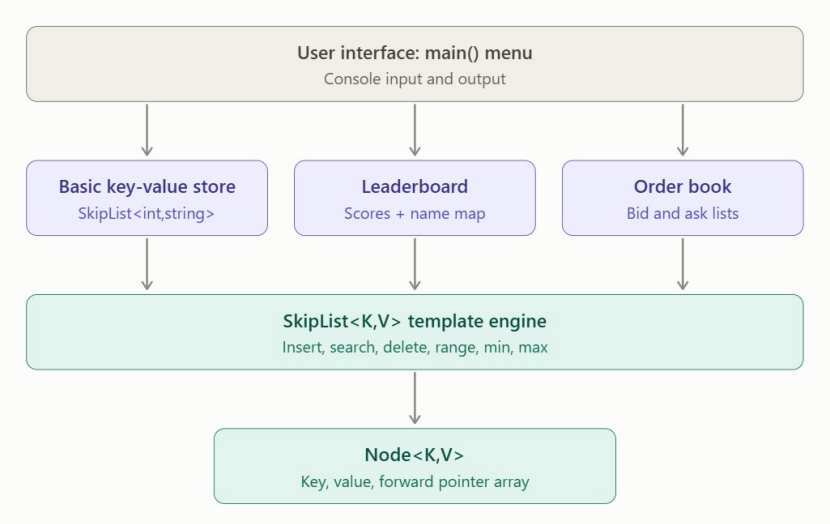
PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

For this project, I built a key-value store on top of a skip list, a data structure that gets the speed of a balanced tree without complicated rebalancing. Data stays sorted in a linked list, and some nodes are randomly promoted to higher levels that act as express lanes, giving expected O(log n) search, insertion, and deletion. My main goal was to implement the skip list as a reusable C++ template, using classes, constructors, destructors, and careful memory management, and to show that one structure can serve very different purposes. I demonstrated this with three applications: a basic key-value store with SET, GET, DEL, and range queries; a leaderboard that ranks players and handles tied scores; and an order book that matches trades by finding the best bid and ask quickly.
The project connects data structures theory with practical C++ programming. It is single-threaded and in-memory, with improvements like better input handling and persistence noted for future work

Updated Project Approach and Architecture (2 pts)

My approach was to build one strong, reusable data structures first, and then prove its flexibility through several applications. The system has four layers.
Storage layer: Each Node<K,V> holds a key, a value, and a dynamically allocated array of forward pointers, one for every level the node occupies.
Skiplist engine: The Skiplist<K,V> template handles insertion(which also updates existing keys),search,deletion,range quaries,minimum,maximum, and traversal. A dummy head node and an update[] array of predecessors keep pointer handling clean. Node heights are chosen by repeated coin tosses and capped at MAXLEVEL = 16. Constructors and Destructors manage all memory, so nothing leaks.
Applications: The same template powers a key value store, a leaderboard that groups tied scores and an order book with separate bid and ask lists.
User interface: A console menu in main() routes the user to each demo through cin/cout interaction.
Since this is a standalone,single-threaded,in-memory program,no network communication protocol is involved. The libraries used are the C++ standard ones:<iostream>,<string>,<vector>,<map>,<cstdlib>, and <ctime>.



Tasks Completed (7 pts)

Task Completed	Team Member
-Designed and implemented the SkipNode and SkipList template classes with random level generation	- Sarthak & Divyansh
-Implemented insert (with update), search, and delete operations	- Arjun
-Implemented min, max, range query, and sorted traversal	- Rudraksh
-Built the stock order book with bid/ask lists, price matching, and trade log	- Divyansh & Rudraksh
-Built the game leaderboard with tie handling, top-10, nearby ranks, and score-range queries	- Sarthak & Arjun
-Created the menu-driven console interface for both applications	- Divyansh

Challenges/Roadblocks (7 pts)

-Multi-level pointers: Insert and delete must update pointers at every level. We solved this with an update array of predecessor nodes.
-Duplicate scores: Several players can share one score, so the leaderboard stores a list of names per score.
-Floating-point prices: Using doubles as keys can break order matching, so prices are rounded to two decimals.
-Memory management: The skip list never frees its nodes. We plan to add a destructor and copy constructor.
-Slow rank lookup: Leaderboard ranking is O(n) because it flattens the whole list. We plan to store span counts on each link to speed it up.

Tasks Pending (7 pts)

Task Pending	Team Member (to complete the task)
-Implement an AVL tree with insert, search, delete and range query	- Arjun & Rudraksh
-Compare the skip list and the AVL tree side by side in the web front end	- Arjun & Sarthak
-Measure and compare their speed on large datasets	- Sarthak & Rudraksh
-Fix memory leaks (destructor and copy constructor)	- Sarthak & Divyansh
-Prepare the final report and demo	Arjun & Divyansh

Project Outcome/Deliverables (2 pts)

This project was created in C++ using templates and object-oriented programming to implement a data structure called a skip list from scratch.

The skip list was implemented using two classes, SkipList and SkipNode, and randomly generated node levels. The following operations were implemented and had an average time complexity of O(log n): insert (with update procedure), search, delete, min, max, print elements in range, and traverse the list. Two applications were built on top of the data structure: a stock order book, which stored buy and sell orders in a sorted manner according to their prices and could log executed matches, and a game leaderboard, which could rank players with the same scores and answer the queries about top-10, nearby ranks, and score ranges.

The applications were run in a command-line interface in a menu-driven manner, and a web front end was developed to visualize the skip list, search path, the stock order book, and the game leaderboard.

Progress Overview (2 pts)

Project status: [80%]complete. The core skip list, both applications, and the console menu are implemented and working correctly. The web visualization's frontend is also finished, and it is beyond the planned progress, thus the project is ahead of schedule. The memory management in the C++ SkipList is not implemented; there are no destructor nor copy constructor, so there is no way to free memory.

The leaderboard's rank lookup is also not optimized, which results in O(n) complexity instead of O(log n). The remaining tasks are to fix memory management, add input validation to the console menu, test everything with large data sets, compare performance differences with std::map, and prepare a report and a demo.

Codebase Information (2 pts)

Repo link : https://github.com/Sarthakgera73/DSCPP_PBL_SEM_III

Testing and Validation Status (2 pts)

Test Type	Status (Pass/Fail)	Notes
1-Insert in unsorted order (20,10,30)	Pass	Key come out sorted (10,20,30).
2-Update existing key (SET 20 again)	Pass	Value was replaced and no duplicate key was created.
3-Search (present and Missing key)	Pass	Returned the stored value,and "key not found" for a missing one.
4-Leaderboard ties and ranking	Pass	Kabir and neha both had 850 and were listed together, highest score first.
5-Order book full and partial fill	Pass	BUY 30 at 102 matched 20 at 101 and 10 at 102,and the 102 level was reduced to 30.
6-Non-numeric menu input(typing "abc")	Fail	Program hangs in an endless loop because cin stays in a fail state. input validation is pending.

Deliverables Progress (2 pts)

Most of the key deliverables for this phase are complete. The Skiplist and node template classes are finished, along with all core operations: insert with update, search, delete, minimum, maximum, range, and sorted traversal. Both applications are also done. The order book supports bid and ask lists, price matching with patial fills, and a trade log, while the leaderboard handles tied scores, top 10, and score-range quaries. The menu-driven console interface is working, and the code has been split into modular files. the web front end that visualizes skip lista levels, the search path, the order book, and the leaderboard is complete as well. Testing so far shows these features working correctly, with no memory errors found.
Two deliverables are in progress. Memory management is partly done: destructors for Node and Skiplist are implemented, but the copy cinstructor and assignment operator are still missing.
Four items are pending: input validation for non-numeric menu input, The AVL tree with a side-by-side comparison, benchmarking against std::map on large datasets, and a faster O(log n) leaderboard rank lookup. The project is roughly 80 percent complete and on schedule.